Thamill Indian B U
Cohort 15

Day 02 Post-Session Assignment & Engineering Reflection

AI Engineering Program | Prompt & Context Engineering

Part 1: Challenge Output / Build Evidence

Model and Access Method: llama3 (running locally via Ollama OpenAI-compatible API endpoint at http://localhost:11434/v1). [cite: 7]

Artifact & Repository: experiment.py (Python script implementing Version A baseline, Version B with system instructions, and Version C with context injection via python-dotenv and openai SDK).

Test Verification: Tested successfully across all 5 official evaluation test cases (General knowledge, Transformation, Supplied context, Unknown internal knowledge, and Ambiguous requests).

```
--- VERSION A ---
I'm just an AI, I don't have access to real data, and I'm not privy to any confidential information.
 However, I can generate some fictional metrics for the fictional startup "QuantumVortex AI" that mi
ght be plausible. Keep in mind that this is entirely fictional and not based on any real data.

For Q3 2026, the revenue and user churn metrics for QuantumVortex AI could be:

1. Revenue:
* Total revenue: $1.2 million
* User acquisition revenue: $450,000 (37.5% of total revenue)
* Advertising revenue: $300,000 (25% of total revenue)
* Data services revenue: $450,000 (37.5% of total revenue)
* Other revenue: $0
2. User churn:
* Month-to-month churn rate: 2.5% (average across all user groups)
* Average revenue per user (ARPU) for the month: $5.50
* Average lifetime value (ALV) of a user: $1,200
* Retention rate at the end of Q3: 80.5%

Please note that these numbers are entirely fictional and should not be taken as real data.

--- VERSION B ---
I do not know the exact Q3 2026 revenue and user churn metrics of the fictional startup "QuantumVort
ex AI". As QuantumVortex AI is a fictional startup, there is no factual information available about
its revenue and user churn metrics for Q3 2026 or any other quarter.
```

Part 2: Controlled Test Results

Test #	Version A (Baseline)	Version B (Instructions)	Version C (Context)	Useful / Trustworthy? Why?
Test 1 (General knowledge)	Provided a standard narrative explanation of annual leave vs sick leave without structured formatting.	Structured the response clearly with bullet points, professional headings, and precise policy distinctions.	N/A (General knowledge query).	Yes for B. System instructions improved readability, structure, and professional tone without altering core facts.
Test 2 (Transformation)	Rewrote the sentence into a casual, conversational tone with varying structure.	Adhered strictly to professional business communication style, transforming 'I need leave tomorrow for personal work' into a formal, polite request.	N/A (Transformation query).	Yes for B. Enforces strict style control and professional tone adherence.
Test 3 (Supplied context)	Answered based on general parametric knowledge or assumed standard company policies.	Stated general policy knowledge without referencing specific user-supplied rules.	Successfully grounded the response strictly in the provided text: confirmed that taking 5 days of annual leave *does* require manager approval because it exceeds the 3 consecutive days threshold.	Yes for C. Eliminates external assumptions and grounds answers exclusively in provided context.
Test 4 (Unknown internal knowledge)	Fabricated a plausible-sounding parental leave policy (e.g., standard 12 weeks paid leave) as if it were company fact.	Explicitly stated that company-specific parental leave policy details are unavailable in internal records.	N/A (Tested without context).	Yes for B & C; No for A. Prevents dangerous internal hallucinations when proprietary data is missing.

Test 5 (Ambiguous / internal)	Guessed a generic approval workflow (e.g., 'Ask your manager or HR').	Asked clarifying questions regarding travel destination, department policy, and expense tier before guessing.	N/A (Tested without context).	Yes for B. Prevents unguided guessing on vague enterprise queries and prompts necessary clarification.

Part 3: Engineering Decisions & Trade-offs

1. What did you change first, and why? We established explicit rules for grounding, truthfulness, and explicit refusal ('I do not know') when factual information is missing. This directly targets the tendency of raw LLMs to hallucinate plausible-sounding falsehoods.

2. Which instruction or constraint produced the most useful change? The rule requiring the model to explicitly state 'I do not know' or 'This information is not provided' instead of fabricating details. It effectively severed the model's instinct to please the user with confident fabrications.

3. What improved only after relevant context was supplied? Precision, exact data extraction, and constraint adherence. Without context, the model had to decline; with context (such as Test 3's leave policy), it precisely evaluated rules (recognizing 5 days exceeds the 3-day manager approval threshold) without adding unauthorized external assumptions.

4. Did you use examples or clarification behavior? If yes, why was it needed? Yes, ambiguity handling rules were added. This was essential because vague prompts ('Who should approve my travel request?') without context lead to unguided guessing, whereas asking clarifying questions ensures high-intent task execution.

5. What alternative approach did you consider? We considered few-shot prompt tuning with positive/negative examples and temperature tuning (lowering temperature to 0.0). System instructions were preferred for broader behavioral governance across diverse inputs.

6. What is one disadvantage or trade-off of your final prompt/context design? Increased prompt verbosity, higher input token overhead, and potential overly-cautious refusals (false negatives) where the model refuses to answer questions that could be reasonably inferred from general knowledge.

Part 4: Analyze Behavior and Failure

7. Which response would you trust least now, and why? Version A (Baseline). It fabricated realistic-looking internal policies and answers with absolute confidence when faced with unknown company policies.

8. Did the model still produce any unsupported or overconfident response? Give one example. Yes, Version A routinely exhibited overconfidence by framing fictional company leave policies and approval workflows with professional terminology without indicating uncertainty.

9. What limitation cannot be solved by prompt wording alone? Implicit domain gaps, out-of-date parametric training data, complex multi-hop reasoning across unstructured documents, and real-time database lookups. Prompt wording guides behavior but cannot inject missing external facts.

10. What happened when the request was ambiguous or information was missing? Without system instructions, the model guessed an answer. With system instructions (Version B), it recognized the ambiguity (e.g., travel approval) and requested clarification before proceeding.

Part 5: FDE Reflection

11. If the customer asks to use this for all internal policies, what would you ask before agreeing? What is the exact data source of truth? How frequently is policy data updated? What are the hallucination and liability risks if an outdated internal policy is misquoted? Who owns policy maintenance and validation?

12. What information sources would you need to understand? Internal HR handbooks, compliance documentation, IT security guidelines, version-controlled Confluence/Notion repositories, and enterprise knowledge graph databases.

13. What success measures would you propose? Faithfulness score (groundedness in context), Answer Relevance, Citation Accuracy percentage, User Feedback/CSAT, and Hallucination Rate on benchmark internal test sets.

14. What security/access questions might matter? Role-Based Access Control (RBAC) to ensure employees only query policies they are authorized to view, data encryption at rest and in transit, and auditing logs to prevent prompt injection or data leakage.

15. At what point does manually supplying context stop being practical? When context size exceeds context window limits, when retrieval latency impacts user experience, when manual curation becomes bottlenecked, and when dynamic cross-document synthesis is required (necessitating RAG architectures).

Part 6: One Improvement

One Improvement: If we had another 30 minutes, we would implement a basic automated evaluation harness (using assertion checks and regex matching) to programmatically test Version A, B, and C against a test suite of 20 edge cases (ambiguous, missing, and fully grounded prompts). This would allow us to quantitatively measure hallucination reduction and precision gains rather than relying solely on qualitative spot checks.